



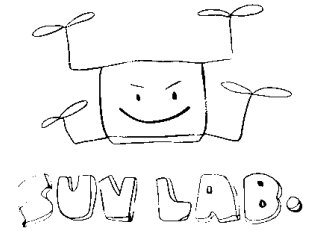
충북대학교

K-뉴딜 아카데미 · AI를 위한 PYTHON · PART 2-3

Pandas

데이터프레임 활용 및 전처리

충북대학교 지능로봇공학과 **문성태** 교수



What is Pandas

NumPy 배열에 이름표를 붙여 표로 다루게 만든 라이브러리

Who

- **Wes McKinney** — 2008년 금융 데이터 분석 도구로 시작
- **공개** — 2009년 오픈소스로 시작 (panel data)
- **지금** — NumFOCUS 후원, BSD 3-Clause

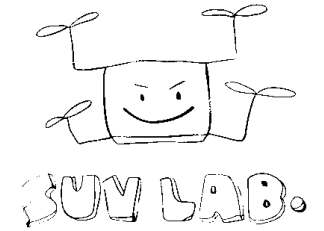
Why

- **문제** — 배열에는 열 이름도 시각도 없음
- **해법** — 인덱스와 열 이름을 붙인 표 자료구조
- **기능** — 결측 처리, 시계열, 그룹 집계 기본 기능

Where

- **입력** — CSV, Excel, SQL, Parquet 을 한 줄로 적재
- **연결** — scikit-learn, statsmodels, 시각화 도구의 공통 입력
- **현장** — 금융, 실험 데이터, 로그와 센서 분석

버전 — 1.0 은 2020년, 2.0 은 2023년. (본 강의는 2.x 기준)



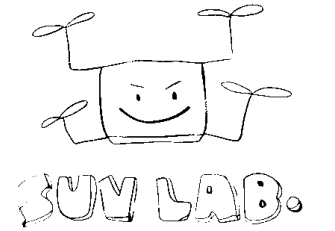
왜 Pandas인가

NumPy 는 숫자 상자, Pandas 는 이름표가 붙은 표

- 얼마다 다른 이름과 dtype
- 결측값(NaN)을 정식 지원
- 시간 인덱스, 문자열, 범주형 지원
- CSV, Excel, Parquet, SQL 직접 입출력
- 내부는 NumPy 배열, 속도를 그대로 계승

```
1 import pandas as pd
2
3 df = pd.read_csv("log.csv")
4 df.head()
5
6 #           t    temp  hum status
7 # 0  0.0000  21.5  40.2      ok
8 # 1  0.0100  21.6  40.1      ok
```

센서 로그, 실험 기록, 설문 응답처럼 **행과 열에 의미가 있는 데이터**가 대상



Series 와 DataFrame

라벨이 붙은 1차원이 Series, 같은 라벨을 공유하는 열의 묶음이 DataFrame

SERIES 1D values with an index

index	temp
t0	21.5
t1	21.6
t2	21.4
t3	21.7

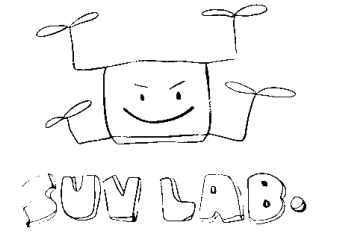
`s["t1"] -> 21.6`

DATAFRAME columns sharing one index
one Series

index	temp	hum	status
t0	21.5	40.2	ok
t1	21.6	40.1	ok
t2	21.4	41.0	warn
t3	21.7	40.8	ok

`df["temp"] -> Series`

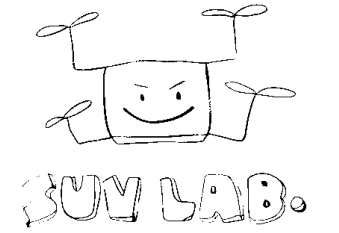
- **Series** — 값 배열 하나에 인덱스가 짝지어진 형태
- **DataFrame** — 인덱스를 공유하는 Series 들의 모음
- **열 선택** — `df["temp"]` 의 결과가 Series
- **정렬 기준** — 연산은 위치가 아니라 인덱스 기준



Series 기본

```
1 s = pd.Series([21.5, 21.6, 21.4],
2               index=["a", "b", "c"])
3
4 s["b"]          # 21.6
5 s.mean()        # 21.5
6 s.index         # Index(['a', 'b', 'c'])
7 s.values        # numpy 배열
```

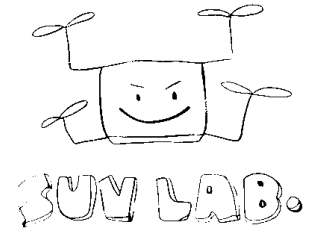
- 1차원 배열 + **인덱스 라벨**
- DataFrame 의 열 하나가 Series
- 인덱스 기준으로 자동 정렬 후 연산
- 이름으로 접근, 위치를 외울 필요 없음



DataFrame 기본

```
1 df = pd.DataFrame({
2     "t": [0.0, 0.1, 0.2],
3     "temp": [21.5, 21.6, 21.4],
4     "status": ["ok", "ok", "warn"],
5 })
6
7 df.columns    # 열 이름
8 df.index      # 행 라벨
9 df.shape      # (3, 3)
```

- 2차원 표, 열 하나하나가 Series
- 열마다 다른 dtype 가능
- index는 기본적으로 0부터의 정수
- 시계열은 시간을 index 로 두는 편이 유리



데이터 불러오기

```
1 df = pd.read_csv(  
2     "log.csv",  
3     sep="," ,  
4     header=0,  
5     parse_dates=["timestamp"],  
6     dtype={"sensor_id": "category"},  
7     na_values=["NA", "-999"],  
8 )
```

sep

구분자, 탭은 \t

names

열 이름 직접 지정

dtype

열별 타입 고정

header

열 이름 행 번호

parse_dates

시간으로 변환할 열

na_values

결측로 간주할 값

센서 로그는 **-999, NA, 빈 문자열** 같은 값이 결측 표시로 쓰인다. 읽는 순간 처리하는 편이 안전.

대용량 읽기 옵션

```
1 pd.read_csv("big.csv",
2             usecols=["t", "ax"])
3 # 필요한 열만
4
5 pd.read_csv("big.csv", nrows=1000)
6 # 앞부분만 미리보기
```

- 열을 줄이는 것이 가장 큰 절감
- 먼저 nrows 로 구조 확인

```
1 reader = pd.read_csv(
2     "big.csv", chunksize=100_000)
3
4 for chunk in reader:
5     process(chunk)
6 # 나누어 처리하면 메모리가 안전
```

- chunksize 는 반복자를 반환
- 반복해 읽는다면 parquet 으로 전환

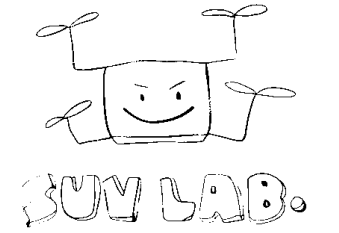
데이터 훑어보기

```
1 df.head()          # 앞 5행
2 df.tail(3)         # 마지막 3행
3 df.sample(5)       # 무작위 5행
4 df.shape           # (행, 열)
5 df.info()          # dtype, 결측, 메모리
6 df.describe()      # 수치 요약
7 df.dtypes          # 열별 타입
```

적재 직후 루틴

- 1 shape 로 크기 확인
- 2 info 로 dtype 과 결측 확인
- 3 describe 로 값 범위 확인

describe에서 **min** 이 **-999**, **max** 가 **99999** 라면 결측 표시이거나 이상치.



열 선택과 추가

```
1 df["temp"]          # Series
2 df[["temp", "hum"]] # DataFrame
3
4 df["temp_f"] = df["temp"] * 9 / 5 + 32
5
6 df = df.drop(columns=["unused"])
7 df = df.rename(columns={"temp": "temp_c"})
```

```
1 # 열 이름 일괄 정리
2 df.columns = df.columns.str.strip().str.lower()
```

- 대괄호 하나는 Series, 둘은 DataFrame
- 열 생성은 바로 대입
- drop 과 rename 은 **새 객체를 반환**
- 열 이름은 소문자와 밑줄로 통일

행 선택: loc와 iloc

loc — 라벨 기준

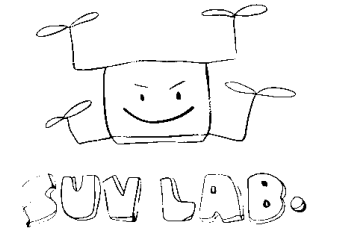
```
1 df.loc[0]
2 df.loc[0:2]    # 0,1,2 끝 포함
3 df.loc[df["temp"] > 30, "temp"]
```

iloc — 위치 기준

```
1 df.iloc[0]
2 df.iloc[0:2]  # 0,1 끝 제외
3 df.iloc[0, 1] # 첫 행 둘째 열
```

loc 슬라이스는 끝을 포함, **iloc** 는 미포함. 파이썬 리스트와 달라 가장 자주 틀리는 지점.

시간을 인덱스로 두면 **df.loc["2026-08-21"]** 처럼 날짜 문자열로도 절단 가능.



조건 필터링

불리언 인덱싱

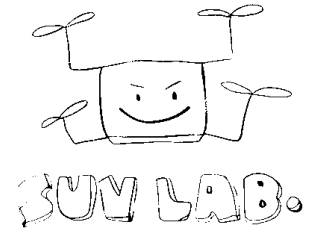
```
1 df[df["temp"] > 30]
2
3 df[(df["temp"] > 30) &
4     (df["hum"] < 50)]
5
6 df[df["status"].isin(["ok", "warn"])]
7
8 df[df["temp"].between(10, 40)]
```

NumPy 와 동일 — **and / or** 대신 **& |** 와 괄호.

query로 읽기 쉽게

```
1 df.query("temp > 30 and hum < 50")
2
3 limit = 30
4 df.query("temp > @limit")
```

조건이 길어지면 query 가 읽기 좋음. 파이썬 변수는 **@이름**으로 전달.



연습문제 1

df 에서 `temp >= 30` 이면서 `hum < 40` 인 행만 골라 몇 행인지 구하시오.

```
1 df.head()
2 df.shape
3
4 # IMPLEMENT HERE
```

df 의 열 — `t` 유닉스 시각(초), `timestamp`, `day`, `sensor_id`, `id`, `temp` 온도, `hum` 습도, `status` . 기대 출력 — 58 (총 602 행 중)

Setting With Copy Warning

사본에 대입하면 원본은 그대로. 경고는 그 사실을 알리는 신호

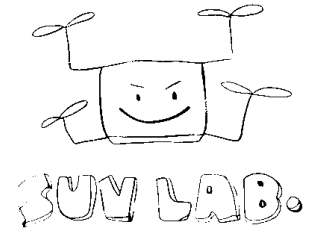
연쇄 인덱싱

```
1 sub = df[df["temp"] > 30]
2 sub["temp"] = 0          # 경고
3
4 df["temp"].max()         # 원본은 그대로
```

loc 로 한 번에

```
1 df.loc[df["temp"] > 30, "temp"] = 0
2
3 df["temp"].max()         # 반영됨
```

- 1 df[조건] 이 뷰인지 사본인지 pandas 도 보장하지 못함
- 2 연쇄 인덱싱은 두 단계. 첫 단계가 사본이면 둘째 단계의 대입은 폐기
- 3 loc 로 행과 열을 한 번에 지정하면 원본을 직접 수정. 사본이 필요하면 .copy()

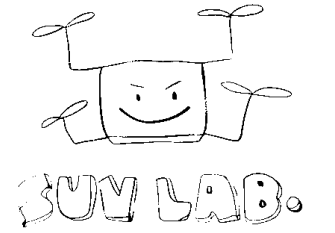


결측값 탐지

```
1 df.isna().sum()
2 # 열별 결측 개수
3
4 df.isna().mean()
5 # 열별 결측 비율
6
7 df.isna().any(axis=1).sum()
8 # 결측을 하나라도 포함한 행 수
```

```
1 # 결측이 있는 행을 직접 들여다본다
2 df[df.isna().any(axis=1)].head()
```

- isna 와 isnull 은 같은 함수
- 결측 **비율**을 먼저 확인. 1% 와 60% 는 대응이 다름
- 열별로 볼지 행별로 볼지 구분
- 결측이 몰린 구간이 있는지 시간축으로 확인



결측값 처리 전략

```
1 df.dropna()
2 df.dropna(subset=["temp"])
3 df.dropna(axis=1, thresh=100)
4
5 df["temp"].fillna(0)
6 df["temp"].ffill()
7 df["temp"].bfill()
8 df["temp"].interpolate()
```

dropna

버려도 될 때

ffill

직전 값을 뒤로 끌기, 상태 유지

interpolate

연속 신호, 선형 보간

fillna

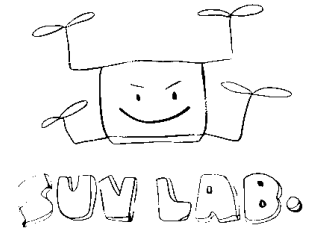
기본값이 명확할 때

bfill

다음 값을 앞으로 당기기, 시작부
결측

방향 — ffill 은 마지막 관측을 이후로 전파, bfill 은 다음 관측을 이전으로 소급. 맨 앞 결측은 ffill 로, 맨 뒤 결측은 bfill 로 채울 수 없음

기본 전략 — 센서 시계열은 보간, 상태 플래그는 ffill, 범주형은 미상 범주 추가

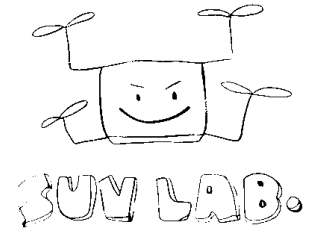


연습문제 2

df 의 temp 열에 결측이 있다. (1) 결측 개수를 세고, (2) 앞 값과 뒤 값으로 채운 뒤, (3) 남은 결측이 0 인지 확인하시오.

```
1 df["temp"].head(12)
2
3 # IMPLEMENT HERE
```

기대 출력 — 채우기 전 결측 8 개, 채운 뒤 0 개 . 주의 — 맨 앞이 결측이면 앞 값이 없다. ffill 다음에 bfill 까지 해야 0 이 된다

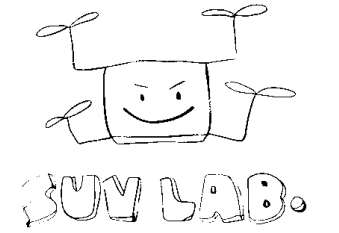


중복 처리

```
1 df.duplicated().sum()
2 # 완전히 같은 행의 개수
3
4 df = df.drop_duplicates()
5 # subset=None, keep="first" 와 같다
6
7 df = df.drop_duplicates(
8     subset=["t"], keep="last")
9 # 같은 시각은 마지막 값만 남긴다
```

- **인자 없이 호출** — subset=None, keep="first" 가 기본값
- subset=None — 모든 열의 값이 같은 행만 중복으로 판단
- keep="first" — 첫 행은 남기고 뒤따르는 중복만 제거
- duplicated() 도 같은 기본값. 첫 행은 False, 이후 중복은 True
- keep 은 first, last, False 중 선택. False 는 중복 행을 전부 제거

센서 로그에서 **같은 타임스탬프가 두 번** 나오면 재전송이거나 버퍼 중복. 지우기 전에 원인 확인.



자료형 정리

```
1 raw = pd.Series(["21.5", "n/a", "-999"])
2 num = pd.to_numeric(raw, errors="coerce")
3 print(num.tolist())
4
5 try:
6     pd.to_numeric(raw, errors="raise")
7 except ValueError as err:
8     print("raise:", err)
```

- **errors="raise"** — 기본값. 숫자로 못 바꾸는 값이 하나라도 있으면 예외
- **errors="coerce"** — 실패한 값만 NaN 으로 바꾸고 나머지는 진행
- **주의** — 센티널 -999 는 숫자라 통과. 범위 검사로 따로 처리

```
1 s = df["status"]
2 c = s.astype("category")
3
4 use = df.memory_usage(deep=True).sum()
5
6 print("object ", s.memory_usage(deep=True))
7 print("category", c.memory_usage(deep=True))
8 print("total MB", round(use / 1e6, 2))
```

- **memory_usage** — 열별 바이트 사용량. sum() 으로 전체 합산
- **deep=True** — 문자열 내용까지 계산. 기본값은 포인터만 세어 과소 집계
- **category** — 반복 문자열을 코드 + 사전으로 저장. 종류가 적을수록 절감 폭이 큼

문자열 처리 (.str)

```
1 s = df["status"]
2
3 s.str.strip()
4 s.str.lower()
5 s.str.contains("err")
6 s.str.startswith("warn")
7 s.str.replace("_", "-", regex=False)
8 s.str.len()
```

```
1 df["code"] = (
2     df["tag"].str.split("-").str[0])
3
4 df["num"] = (
5     df["raw"].str.extract(r"(\d+)"))
```

.str 접근자로 파이썬 문자열 메서드를 열 전체에 한 번에 적용. 결측은 자동으로 건너뛴

센서 로그의 **앞뒤 공백과 대소문자 혼재**가 가장 흔한 오염. 적재 직후 strip 과 lower 로 통일

날짜 시간 처리

```
1 df["t"] = pd.to_datetime(df["t"])
2
3 df["t"].dt.year
4 df["t"].dt.hour
5 df["t"].dt.dayofweek
6 df["t"].dt.floor("1min")
7
8 df = df.set_index("t").sort_index()
9 df.loc["2026-08-21"]
```

- 문자열을 **datetime64**로 바꿔야 시간 기능 사용 가능
- .dt 접근자로 연, 월, 시, 요일 추출
- 시간을 index 로 두면 구간 자르기가 쉬움
- 리샘플링은 반드시 **정렬** 후에

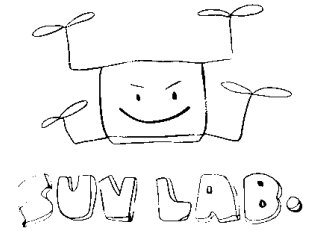
```
1 # 유닉스 시각 입력
2 df["t"] = pd.to_datetime(
3     df["t"], unit="s", utc=True)
```


연습문제 3

`t` 는 유닉스 시각(초)이다. 이를 `datetime` 으로 바꿔 인덱스로 두고 정렬한 뒤, **2026-08-22 하루치만** 잘라 행 수와 `temp` 평균을 구하시오.

```
1 df[["t", "temp"]].head()
2
3 # 1. t 를 datetime 으로 바꾼다
4 # 2. t 를 인덱스로 두고 정렬한다
5 # 3. 2026-08-22 하루치만 day 에 담는다
6 # IMPLEMENT HERE
7
8 len(day), round(day["temp"].mean(), 2)
```

쓸 함수 — `pd.to_datetime(..., unit="s")`, `set_index`, `sort_index`, `loc["날짜"]` · **데이터** — 2026-08-21 00:00 부터 7분
간격 602행, 약 3일치 · **기대 출력** — (206, 27.85)



정렬과 순위

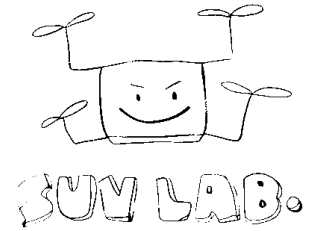
정렬은 새 객체를 반환, 순위는 동점 처리 규칙이 핵심

```
1 top = df.sort_values(["day", "temp"],
2                       ascending=[True, False])
3 print(top[["day", "temp"]].head(3).to_string())
4
5 t = pd.Series([21.5, 21.6, 21.5])
6 for m in ("average", "min", "dense"):
7     print(m, t.rank(method=m).tolist())
```

값	average	min	dense
21.5	1.5	1	1
21.6	3	3	2
21.5	1.5	1	1

- **sort_values** — by, ascending 을 리스트로 주면 열마다 방향 지정
- **na_position** — 결측 위치 선택. 기본은 last

- **rank(method)** — average 나눔, min 낮은 순위, dense 연속
- **nlargest(n)** — 상위 n 개만 추출. 인덱스는 원본 유지



apply와 map

```

1 s = df["status"].map({"ok": 0, "warn": 1})
2 print(s.head(4).tolist(), int(s.isna().sum()))
3
4 r = df.apply(lambda x: x["a"] + x["b"], axis=1)
5 print(r.head(3).tolist())
6
7 m = df[["a", "b"]].apply(lambda c: c.max())
8 print(m.to_dict())

```

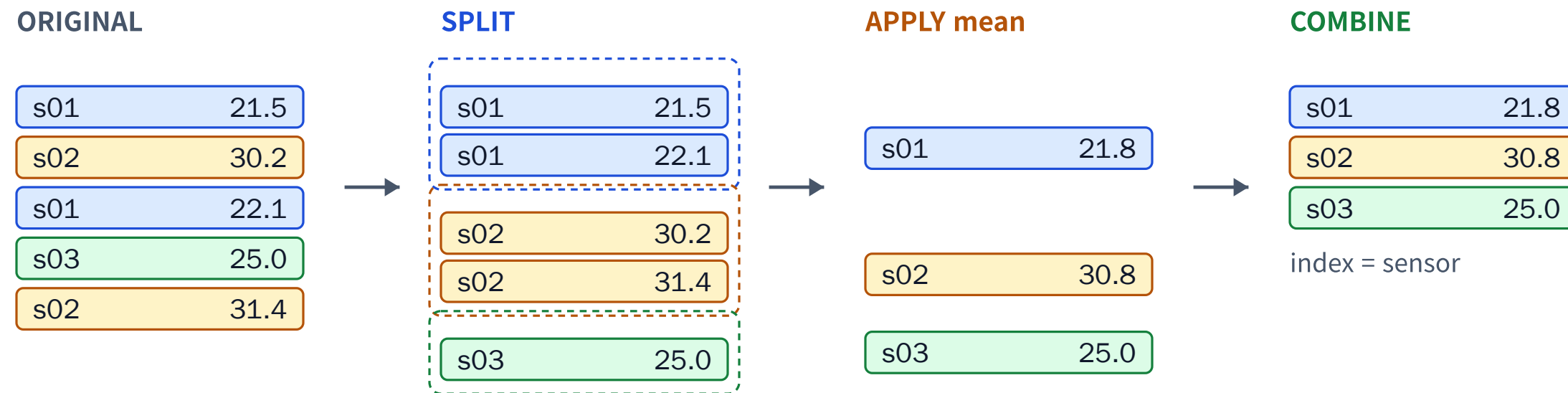
호출	전달 단위	반환
s.map(dict)	원소 하나	Series
df.apply(f, axis=1)	행 Series	Series
df.map(f)	원소 하나	DataFrame

map 은 Series 전용, apply 는 축 단위, df.map 은 전 원소 단위

- **map** — 함수, dict, Series 를 인자로 받음. 대응 키가 없으면 NaN
- **apply** — axis=0 은 열 Series, axis=1 은 행 Series 를 함수에 전달
- **DataFrame.map** — 전 원소에 적용. pandas 2.1 이전 이름은 applymap
- **성능** — apply 는 파이썬 루프. 벡터 연산, 내장 메서드, map, apply 순으로 검토

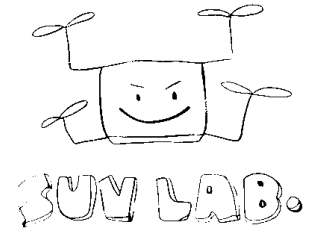
groupby 개념

나누고(split), 각각 계산하고(apply), 다시 합친다(combine).



왼쪽 원본을 범주별로 나누고(split), 그룹마다 계산해(apply), 한 표로 합친다(combine).

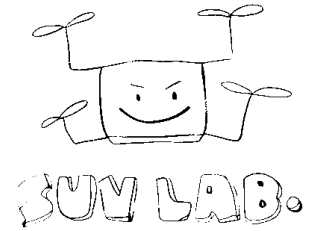
```
1 df.groupby("sensor_id")["temp"].mean()      # 센서별 평균 온도
2 df.groupby(["day", "sensor_id"]).size()     # 날짜별 센서별 샘플 수
```



groupby 집계

```
1 summary = df.groupby("sensor_id").agg(  
2     temp_mean=("temp", "mean"),  
3     temp_max=("temp", "max"),  
4     temp_std=("temp", "std"),  
5     n=("temp", "size"),  
6 ).reset_index()
```

- 이름 붙은 집계로 **결과 열 이름을 직접 지정**
- 문자열 대신 함수 전달 가능
- reset_index 로 그룹 키를 다시 열로 환원
- size 는 결측 포함, count 는 결측 제외

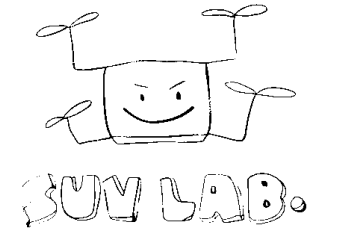


연습문제 4

`sensor_id` 로 묶어 **센서별 평균 온도와 표본 수**를 한 표로 구하고, 평균이 가장 높은 센서의 이름을 찾으시오.

```
1 df[["sensor_id", "temp"]].head()
2
3 # IMPLEMENT HERE
```

센서 — s01, s02, s03 세 종류 · **기대 출력** — 평균과 개수 두 열을 가진 표, 그리고 가장 높은 센서 `s01` (평균 27.48, 188개)



transform과 filter

그룹 연산의 세 갈래 — agg, transform, filter

transform — 행 수 유지

```
1 g = df.groupby("sensor_id")["temp"]
2 z = (df["temp"] - g.transform("mean"))
3 z = z / g.transform("std")
4 print(z.head(3).round(2).tolist())
```

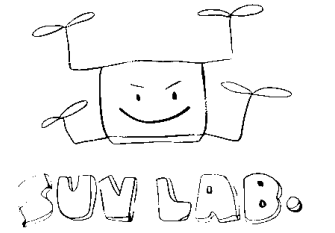
$$z_i = \frac{x_i - \mu_g}{\sigma_g}$$

그룹 평균과 표준편차를 각 행에 되돌려 센서별 z 점수 계산

filter — 그룹 통째 선별

```
1 sz = df["sensor_id"].value_counts()
2 big = df.groupby("sensor_id").filter(
3     lambda d: len(d) >= 200)
4 print(sz.to_dict(), len(df), len(big))
```

- 인자는 그룹 DataFrame 전체
- 반환은 True 또는 False 하나
- 행 수는 줄고 원본 인덱스는 유지



피벗과 재구조화

```
1 pd.pivot_table(df,  
2     index="day",  
3     columns="sensor_id",  
4     values="temp",  
5     aggfunc="mean")  
6 # wide: 행=날짜, 열=센서
```

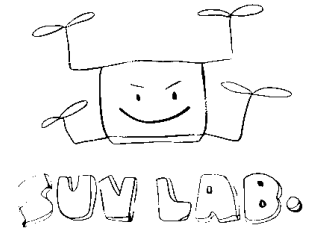
wide 형식

사람이 보기 좋고 보고서에 바로 사용. 대신 열이 계속 증가

```
1 df.melt(  
2     id_vars=["t"],  
3     var_name="channel",  
4     value_name="value")  
5 # long: t, channel, value
```

long 형식

그룹 연산과 시각화에 유리. 채널이 늘어도 구조 유지



데이터 결합

```
1 pd.concat([df1, df2], axis=0)
2 # 같은 구조를 아래로 이어붙인다
3
4 pd.concat([df1, df2], axis=1)
5 # 인덱스를 맞춰 옆으로 붙인다
6
7 df.merge(meta, on="sensor_id",
8          how="left")
```

inner

양쪽에 다 있는 키

left

왼쪽 기준, 가장 흔함

right

오른쪽 기준

outer

합집합, 빈칸은 NaN

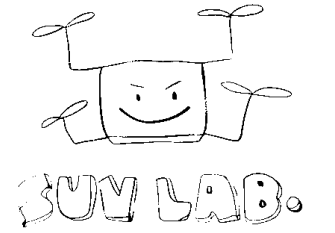
결합 전에 **키 중복 여부**를 확인 필수. 중복 키는 행 수를 폭증. **validate="m:1"**로 자동 검사 가능

시계열 리샘플링

시간 인덱스를 정해진 간격으로 다시 잘라 구간마다 집계

```
1 df = df.set_index("t").sort_index()
2
3 df["temp"].resample("1min").mean()
4 # 다운샘플링: 줄여서 집계
5
6 df["temp"].resample("1s").interpolate()
7 # 업샘플링: 늘려서 보간
8
9 df["temp"].resample("1h").agg(
10     ["mean", "max", "count"])
```

- **무엇** — 시간 간격으로 묶는 groupby
- **왜** — 흔들리는 로그 간격을 균일하게
- **다운샘플링** — 줄여서 mean, max
- **업샘플링** — 늘려서 interpolate
- **전제** — 시간 인덱스와 정렬



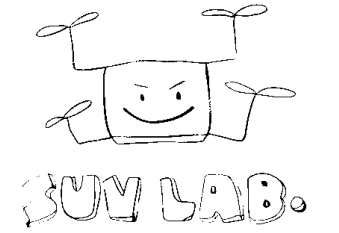
롤링 윈도우

$$\bar{x}_t = \frac{1}{w} \sum_{k=0}^{w-1} x_{t-k}$$

```
1 df["ma"] = df["temp"].rolling(  
2     5, min_periods=1).mean()  
3 # 5샘플 이동평균  
4  
5 df["sd"] = df["temp"].rolling(30).std()  
6 # 이동 표준편차, 이상 감지용  
7  
8 df["ewm"] = df["temp"].ewm(span=10).mean()  
9 # 지수 가중 이동평균
```

- 왜 — 연속한 w 개를 평균 내 잡음을 걷음
- 이상 감지 — 이동 표준편차가 커지면 신호
- resample 과 차이 — 행 수는 그대로
- 창 크기 — 100Hz에서 1초면 100
- min_periods 없으면 앞부분이 NaN

이동평균은 **지연을 만든다는 점에 주의**. 실시간 제어에 쓸 때는 지연량을 계산해 보상



연습문제 5

시간 인덱스를 가진 df 에서 temp 의 1시간 평균 리샘플링 값과 5샘플 이동평균 값을 각각 구해 앞 5개를 비교하시오.

```
1 df.index[:3]
2 df["temp"].head()
3
4 # IMPLEMENT HERE
```

데이터 간격 — 7분. 리샘플링 주기는 이보다 커야 의미가 있다 (1분으로 하면 대부분 결측이 된다) . **기대 출력** — 리샘플링은 600행이 70행으로 줄고 인덱스가 정각으로 바뀐다. 이동평균은 행 수 그대로, 앞 4개는 창이 덜 차서 결측

결과 저장

```
1 df.to_csv("clean.csv", index=False)
2
3 df.to_parquet("clean.parquet")
4
5 df.to_excel("report.xlsx", index=False)
6
7 df.to_json("clean.json",
8           orient="records")
```

index=False를 잊지 않는다

```
1 # index=True (기본값)
2 ,t,temp
3 0,0.0,21.5
4 # 이름 없는 열이 생긴다
```

다시 읽을 때 **Unnamed: 0** 열이 생기는 원인

고르는 기준 — 사람이 열어볼 산출물은 CSV나 Excel, 중간 결과와 대용량은 parquet. 이유는 다음 두 장

CSV 와 Parquet

행으로 쌓는 CSV, 열로 묶는 Parquet

ROW STORE CSV: one record after another

t0	21.5	40.2	ok	t1	21.6	40.1	ok	t2	21.4	41.0	warn
----	------	------	----	----	------	------	----	----	------	------	------

한 열만 필요해도 모든 바이트를 지나간다

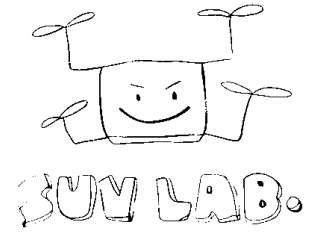
COLUMN STORE Parquet: one column at a time

t0	t1	t2	21.5	21.6	21.4	40.2	40.1	41.0	ok	ok	warn
t			temp			hum			status		

필요한 열 덩어리만 읽는다. 같은 열은 값이 닮아 잘 압축된다

- **CSV** — 한 줄이 한 행. 열 하나만 필요해도 전부 스캔
- **Parquet** — 열 덩어리로 저장. 쓰는 열만 적재

- **압축** — 같은 열은 값이 닮아 압축률이 높음
- **타입** — 스키마를 파일에 함께 저장



Parquet 다루기

```
1 df.to_parquet("clean.parquet")
2 df.to_parquet("c.parquet",
3               compression="zstd")
4
5 pd.read_parquet("clean.parquet")
6 pd.read_parquet("clean.parquet",
7               columns=["t", "temp"])
```

이럴 때 쓴다

- **중간 결과** — 정제 단계 사이에 주고받는 파일
- **대용량** — 열 일부만 반복해서 읽는 분석

형식	크기	읽기 (전체 / 두 열)	타입
CSV	5.4 MB	36 ms / 36 ms	유실
Parquet	3.2 MB	25 ms / 3 ms	보존
Parquet, float32	2.2 MB	-	보존

6만 행 10열 로그 기준. 열이 많고 행이 길수록 격차 확대

비용

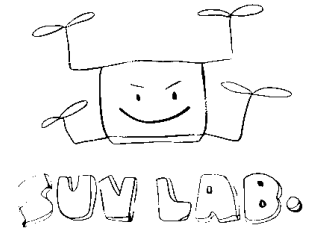
- **이진 형식** — 편집기로 열 수 없고 diff 도 불가
- **엔진 필요** — `pip install pyarrow`

빠른 시각화

```
1 df["temp"].plot()
2 # 선 그래프
3
4 df["temp"].hist(bins=30)
5 # 분포 확인
6
7 df.plot(x="t", y=["ax", "ay", "az"])
8 # 다중 선 그래프
9
10 df.plot.scatter(x="temp", y="hum")
```

- 내부적으로 matplotlib 사용
- 탐색 단계에서 **빠르게 한번 보는 용도**
- 보고서용 그림은 matplotlib 직접 사용
- 이상치는 숫자보다 그림에서 먼저 보임

전처리를 끝냈다면 **한 번 그려 볼 것**. 통계값이 멀쩡해도 그림에서 문제가 드러남



실습 안내 (lab04)

- 1 로그 CSV 적재와 품질 확인
- 2 열 이름, 자료형, 시간 인덱스 정리
- 3 결측과 중복 처리
- 4 센서별 groupby 통계표 만들기
- 5 1분 단위 리샘플링과 이동평균
- 6 정제 결과 저장

체크포인트

- 체인 인덱싱 경고 없이 돌아가는가
- 결측 처리 방식을 설명할 수 있는가
- 저장 후 다시 읽어도 같은 형태인가

30분[lab04-pandas.md](#)

다음: **Part 2-4 센서 데이터 로그 분석 종합 실습** — NumPy 와 Pandas 를 한 파이프라인으로 결합